Rust - Trait Clone

Définition et rôle

En Rust, `Clone` est un trait (interface) standard défini dans le module `std::clone`. Sa seule méthode requise est :

```
fn clone(&self) -> Self;
```

- `&self` : Prend une référence *immuable* à l'instance courante. Immuable parce que `clone()` ne doit *jamais modifier* l'objet original. Il crée une *copie indépendante* (ou partagée, selon le type).
- `-> Self` : Retourne une *nouvelle instance* du même type.

Le trait `Clone` permet de *dupliquer explicitement une valeur* en appelant la méthode `.clone()`.

- *Explicite* : Le développeur doit appeler `.clone()` pour dupliquer une valeur. Rust veut éviter les copies accidentelles coûteuses (ex: cloner un gros `Vec` sans s'en rendre compte). Copie rendue explicite dans le code.
- *Contrôlé* : Le comportement de `clone()` est *déterministe* et dépend du type (primitif, référence, pointeur), c'est-à-dire que le type détermine comment la copie doit être réalisée (prévisibilité et sûreté).

Comportement de `clone()` selon le type

Le comportement de `clone()` varie selon la sémantique du type sous-jacent.

Cas	Qui contrôle `clone()` ?	Exemple
Type primitif (`i32`)	Compilateur (implémentation par défaut)	`5.clone()` → copie bit à bit.
`String` (données dans la *heap*)	Bibliothèque standard	`s.clone()` → nouvelle allocation.
Struct personnalisée	Développeur (via `impl Clone`)	`ma_struct.clone()` → logique custom.

Exemple de dépendance au type (`String` *vs* `&str`):

```rust
// String : copie profonde (nouvelle allocation)
let s1 = String::from("hello");
let s2 = s1.clone(); // Nouvelle String en mémoire
assert_eq!(s1.as_ptr(), s2.as_ptr()); // ❌ Faux (adresses différentes)

// &str : copie superficielle (nouvelle référence)
let s3: &str = "hello";
let s4 = s3; // Pas besoin de .clone() (Copy est implémenté)
assert_eq!(s3.as_ptr(), s4.as_ptr()); // ✅ Vrai (même adresse)
```

Exemple avec une structure personnalisée :

```rust
#[derive(Debug)]
struct Book {
    title: String,
    author: String,
    id: u32,
}

// Implémentation manuelle de Clone()
impl Clone for Book {
    fn clone(&self) -> Self {
        Book {
            title: self.title.clone(),   // Clone la String
            author: self.author.clone(), // Clone la String
            id: self.id,                 // Copy le u32
        }
    }
}

fn main() {
    let book1 = Book {
        title: String::from("Pour Qui Sonne Le Glas"),
        author: String::from("Ernest Hemingway"),
        id: 1,
    };
    let book2 = book1.clone();
    println!("{:?}", book2); // Book { title: "Pour Qui Sonne Le Glas",
author: "Ernest Hemingway", id: 1 }
```

Liste des types qui implémentent `.Clone()` : https://doc.rust-lang.org/stable/std/clone/trait.Clone.html#implementors

A savoir que si nous essayons d'appeler `.clone()` sur un type qui ne l'implémente pas, Rust générera une erreur.

Types à copie profonde (Deep Copy)

Une *copie profonde* crée une *nouvelle allocation mémoire* avec une *copie complète des données*. Les modifications sur la copie *n'affectent pas* l'original.

Types concernés :

- Types primitifs : `i8`, `i16`, `i32`, `i64`, `u8`, `u16`, `u32`, `u64`, `f32`, `f64`, `bool`, `char`. Types dits *triviaux* car ne nécessitent pas de pointeur ou de ressource externe. Leur copie est donc gratuite et instantanée.
- Types possédés : `String`, `Vec<T>`, `HashMap<K, V>`, `Box<T>` (si `T: Clone`).
- Structs / Enums : Si tous leurs champs implémentent `Clone` (via `#[derive(Clone)]`).

Exemple avec String :

```
let s1 = String::from("Hello");
let s2 = s1.clone(); // Nouvelle allocation mémoire pour "Hello"

// Modification de s2 n'affecte pas s1
let mut s2 = s2;
s2.push_str(", world!");
println!("s1: {}", s1); // "Hello"
println!("s2: {}", s2); // "Hello, world!"
```

Nouvelle allocation :

- `String` est un *type possédé* qui gère une chaîne de caractères en mémoire dynamique.
- `clone()` alloue un *nouveau buffer* et copie les octets de `s1` vers `s2`.

Types à copie superficielle (Shallow Copy)

Une *copie superficielle* ne duplique *pas les données sous-jacentes*, mais crée une *nouvelle référence* vers ces données. Les modifications (si possibles) *sont visibles* via toutes les copies.

Types concernés :

- Références immuables (`&T`) : `clone()` retourne une *nouvelle référence* vers `T` . *Aucune allocation mémoire* (car une référence est juste un *pointeur*).
- Pointeurs intelligents : Structures gérant la mémoire automatiquement (ex: `Box` , `Rc` , `Arc`) pour posséder ou partager des données avec des règles de sécurité.

Exemple avec une référence :

```
let x = 42;
let ref_x = &x;
let ref_x_clone = ref_x; // Pas besoin de .clone() (car &T implémente
Copy)

// ref_x et ref_x_clone pointent vers la même adresse
assert_eq!(ref_x as *const i32, ref_x_clone as *const i32); // ✅ Même
adresse
```

Types non-clonables

Certains types *n'implémentent pas* `Clone` car ils représentent des *ressources uniques*, comme les descripteurs de fichier unique (non dupliquables) ou les flux d'entrée standard (ressource système unique).

Les références mutables (`&mut T`) également. Cloner une `&mut T` permettrait d'avoir *deux références mutables* vers `T` , ce qui violerait les règles d'emprunt de Rust .

Différence entre `Clone` et Copy

Critère	Clone	Copy
Mécanisme	Explicite (`x.clone()`)	Implicite (copie automatique)
Coût	Peut être *coûteux* (allocation)	*Gratuit* (copie bit à bit)
Personnalisation	Oui (implémentation manuelle)	Non (comportement fixe)
Types concernés	Tous les types (si implémenté)	Seulement certains types
Sémantique	Duplication logique	Duplication physique (*bitwise*)
Exemples	`String` , `Vec` , `Arc`	`i32` , `bool` , `&T` , tuples de `Copy`

Critère	Clone	Copy
Héritage	`Copy` est un *supertrait* de `Clone`	—

En Rust, tout type implémentant `Copy` doit aussi implémenter `Clone`. `Copy` est une *optimisation* pour les types qui peuvent être copiés *sans code supplémentaire* (ex: `i32`). `Clone` est *plus général* : il permet de dupliquer n'importe quel type, y compris ceux qui nécessitent une logique personnalisée. Par conséquent si un type est `Copy`, alors `x.clone()` doit être *équivalent à* `x` (copie bit à bit).

```
let x: i32 = 5;
let y = x;           // Copy implicite (pas de .clone() nécessaire)
let z = x.clone();   // Clone explicite (équivalent à y)
assert_eq!(y, z);    // true
```

Utiliser `Clone` *vs* `Copy` :

Cas d'usage	Clone	Copy
Dupliquer une `String`	✅ Oui	❌ Non
Dupliquer un `i32`	✅ Oui	✅ Oui
Dupliquer un `Vec`	✅ Oui	❌ Non
Dupliquer une référence (`&T`)	✅ Oui	✅ Oui
Passer un argument à une fonction	❌ Non	✅ Oui
Retourner une valeur depuis une fonction	❌ Non	✅ Oui

Règle pratique

- Utiliser `Copy` pour les types *petits et triviaux* (ex: entiers, booléens).
- Utiliser `Clone` pour les types *complexes ou dynamiques* (ex: `String`, `Vec`).

- Préférer `Clone` ? :
  - Si une fonction a besoin de *conserver une copie* d'un argument (ex: cache).
  - Si nous travaillons avec des *structures de données partagées* .
- Eviter `Clone` :
  - Si la copie est *trop coûteuse* (préférer alors des références).

- Si le type représente une *ressource unique* (type non-clonable).

Méthode `clone_from()`

```
fn clone_from(&mut self, source: &Self) {
    *self = source.clone();
}
```

- `&mut self` : Prend une référence mutable à l'instance courante. `clone_from` peut modifier `self` pour réutiliser ses ressources.
- `source: &Self` : Prend une référence immuable à la source à cloner.
- Comportement par défaut : `*self = source.clone()` (remplace `self` par une copie de `source`).

Cette méthode est fournie par défaut, car Rust propose une implémentation générique qui fonctionne pour tous les types, mais qui peut être surchargée pour des optimisations. Cela évite une allocation inutile si `self` a déjà assez de capacité. Elle s'avère plus rapide que `clone()` et est personnalisable pour des types complexes.

Cas d'usage :

Type	Avantage de `clone_from`	Exemple
`Vec`	Réutilise la capacité allouée de `self` si elle est suffisante.	`vec1.clone_from(&vec2);`
`String`	Réutilise le *buffer* de `self` si sa capacité est suffisante.	`s1.clone_from(&s2);`
`HashMap`	Réutilise la *table de hachage* existante de `self`.	`map1.clone_from(&map2);`
Types personnalisés	Permet une *logique de copie optimisée* (ex: réinitialiser des champs avant de copier).	`my_struct.clone_from(&other);`

Exemple avec `Vec<T>` (comparatif entre `clone()` et `clone_from`) :

```
use std::time::Instant;

fn main() {
    let mut vec1 = vec![0; 10_000_000];
    let vec2 = vec![0; 10_000_000];
```

```rust
    // Méthode 1: clone()
    let start = Instant::now();
    let vec_clone = vec1.clone();
    let temps_clonage = start.elapsed();

    // Méthode 2: clone_from
    let start = Instant::now();
    vec1.clone_from(&vec2);
    let temps_from_clone = start.elapsed();

    println!("clone(): {temps_clonage:?}"); // 50.831921ms (dépend du système)
    println!("clone_from: {temps_from_clone:?}"); // 37.032039ms (dépend du système)
}
```

Si `vec1` a une capacité supérieure ou égale à celle de `vec2`, `clone_from` *ne réalloue pas* de mémoire (plus optimisé). Sinon, il *agrandit* `vec1` juste assez pour contenir `vec2`. `clone_from` commence par comparer `vec1.capacity()` et `vec2.len()` avant de réaliser le clonage. Avec `clone()`, l'allocation mémoire est systématique, donc moins optimisé.

Ainsi, nous allons préférer utiliser `clone_from` quand nous voulons réutiliser une instance existante, et `clone()` quand nous souhaitons obtenir une instance indépendante.

Voir clone_from

📚 Pour aller plus loin

- Documentation officielle de `Clone`
- Documentation officielle de `Copy`
- A propos des traits dans le Rust Book
- Mastering `Copy` and `Clone` traits in Rust